RISC0 동작 개념 및 구현 분석

목차

1. 개요
2. 핵심 개념
3. 아키텍처 구조
4. 실행 환경 구성
5. 실행 프로세스
6. 세그먼트 관리
7. 증명 시스템
8. 회로 구현
9. 호스트-게스트 통신
10. 보안 모델
11. 성능 최적화
12. 실제 구현 세부사항

---

1. 개요

RISC0는 **zk-STARK 기반의 영지식 증명 가능한 범용 컴퓨팅 플랫폼**입니다. RISC-V 마이크로아키텍처를 기반으로 하여 임의의 코드를 실행하고, 그 실행이 올바르게 수행되었음을 영지식으로 증명할 수 있습니다.

1.1 핵심 특징

- **범용성**: RISC-V를 지원하는 모든 언어(Rust, C, C++) 지원
- **영지식**: 입력 데이터나 실행 과정을 노출하지 않고 결과만 증명
- **검증 가능성**: 누구나 증명을 검증할 수 있음
- **확장성**: 3계층 재귀 증명 시스템으로 확장성 확보

---

2. 핵심 개념

2.1 용어 정의

2.1.1 Method (메서드)

- **정의**: 증명하고자 하는 코드를 컴파일한 RISC-V ELF 파일
- **구성요소**:
- 실행 가능한 바이너리 코드
- 특별한 진입점(entry point)
- Image ID (ELF 파일의 암호학적 해시)

2.1.2 Host와 Guest

- **Host**: zkVM을 실행하고 증명을 생성하는 주체
- **Guest**: zkVM 내부에서 실행되는 논리적 RISC-V 머신
- **통신**: Host는 Guest의 실행을 관찰할 수 있지만 수정할 수 없음

2.1.3 Journal (저널)

- **정의**: Guest 코드가 작성하는 append-only 로그
- **용도**: 계산의 공식적인 출력을 나타냄

- **특징**: 검증자가 확인할 수 있는 유일한 실행 결과

2.1.4 Receipt (영수증)

- **구성요소**:
- **Journal**: 실행 중 작성된 로그
- **Seal**: 불투명한 암호학적 데이터 블롭
- **기능**: 올바른 실행의 영지식 증명

---

3. 아키텍처 구조

3.1 전체 시스템 구조

```
┌─────────────────────────────────────────────────────────────┐
│                          RISC0 시스템                        │
├─────────────────────────────────────────────────────────────┤
│                       Application Layer                      │
│        ┌──────────────────┐        ┌──────────────────┐     │
│        │   Host Program   │        │  Guest Program   │     │
│        │     (Prover)     │        │     (Method)     │     │
│        └──────────────────┘        └──────────────────┘     │
├─────────────────────────────────────────────────────────────┤
│                           zkVM Layer                         │
│        ┌──────────────────┐        ┌──────────────────┐     │
│        │    Executor      │        │       Prover     │     │
```

```
│          │    (실행 엔진)   │         │    (증명 생성)   │       │
│          └──────────────────┘         └──────────────────┘       │
├──────────────────────────────────────────────────────────────────┤
│                          Circuit Layer                           │
│          ┌──────────────────┐         ┌──────────────────┐       │
│          │      RV32IM      │         │     Recursion    │       │
│          │      Circuit     │         │      Circuit     │       │
│          └──────────────────┘         └──────────────────┘       │
├──────────────────────────────────────────────────────────────────┤
│                            ZKP Layer                             │
│          ┌──────────────────┐          ┌──────────────────┐      │
│          │      STARK       │          │      Groth16     │      │
│          │     Protocol     │          │      Protocol    │      │
│          └──────────────────┘          └──────────────────┘      │
└──────────────────────────────────────────────────────────────────┘
```

3.2 주요 컴포넌트

3.2.1 risc0-zkvm

- **역할**: 메인 zkVM 인터페이스
- **기능**:
- Guest 프로그램 실행
- 증명 생성 및 검증

- Host-Guest 통신 관리

3.2.2 risc0-zkp

- **역할**: 영지식 증명 시스템 코어
- **기능**:
- STARK 프로토콜 구현
- 암호학적 해시 함수
- 증명 생성/검증 알고리즘

3.2.3 risc0-circuit-rv32im

- **역할**: RISC-V RV32IM 명령어 세트 회로
- **기능**:
- 32비트 정수 연산
- 메모리 접근
- 제어 흐름

3.2.4 risc0-circuit-recursion

- **역할**: 재귀 증명을 위한 회로
- **기능**:
- 증명 압축
- 다중 세그먼트 결합
- 확장성 제공

---

4. 실행 프로세스

4.1 전체 실행 흐름

```
┌──────────────┐   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐
| 1. 컴파일    |->| 2. 실행     |->| 3. 증명      |->| 4. 검증     |
| (Method)    |  | (Execution) | | (Proving)   |   |(Verification)|
└──────────────┘   └──────────────┘  └──────────────┘   └──────────────┘
```

4.2 단계별 상세 분석

4.2.1 컴파일 단계 (Method Creation)

```rust
// 1. Rust 코드를 RISC-V ELF로 컴파일
// guest/src/main.rs
use risc0_zkvm::guest::env;

fn main() {
let input: u32 = env::read();
let result = fibonacci(input);
env::commit(&result);
}

// 2. Image ID 계산
let image_id = compute_image_id(&elf_bytes);
```

4.2.2 실행 단계 (Execution Phase)

```
// ExecutorImpl 구조체가 실행을 담당
pub struct ExecutorImpl<'a> {
env: ExecutorEnv<'a>, // 실행 환경
image: MemoryImage, // 메모리 이미지
syscall_table: SyscallTable<'a>, // 시스템 콜 테이블
elf: Option<Vec<u8>>, // ELF 바이너리
}

// 실행 과정
impl ExecutorImpl {
pub fn run(&mut self) -> Result<Session> {
// 1. 세그먼트 제한 설정
let segment_limit_po2 = self.env.segment_limit_po2
.unwrap_or(DEFAULT_SEGMENT_LIMIT_PO2 as u32) as usize;

// 2. Executor 생성 및 실행
let mut exec = Executor::new(
self.image.clone(),
self,
self.env.input_digest,
self.env.trace.clone(),
```

```
self.env.povw_job_id,
);

// 3. 세그먼트별 실행
let  result = exec.run(segment_limit_po2, max_insn_cycles, session_limit, |inner
// 각 세그먼트 처리
let  segment = Segment {
index: inner.index as u32,
inner,
output,
};
// 세그먼트 저장
let  segment_ref = callback(segment)?;
refs.push(segment_ref);
Ok(())
})?;

// 4. Session 생성
Ok(Session { /* ... */ })
}
}
```

4.2.3 증명 단계 (Proving Phase)

```
// 3계층 증명 시스템
// Layer 1: Segment Proofs (STARK)
// Layer 2: Recursion Proofs (STARK)
// Layer 3: Groth16 Proof (SNARK)

pub fn prove_session(
session: &Session,
prover_opts: &ProverOpts,
) -> Result<Receipt> {
// 1. 각 세그먼트에 대한 STARK 증명 생성
let segment_receipts: Vec<SegmentReceipt> = session.segments
.iter()
.map(|segment| prove_segment(segment))
.collect::<Result<Vec<_>>>()?;

// 2. 재귀적으로 증명들을 결합
let composite_receipt = prove_recursion(segment_receipts)?;

// 3. 최종 Groth16 증명 생성 (선택적)
if prover_opts.receipt_kind == ReceiptKind::Groth16 {
let groth16_receipt = prove_groth16(composite_receipt)?;
Ok(Receipt::Groth16(groth16_receipt))
```

```
} else {
Ok(Receipt::Composite(composite_receipt))
}
}
```

4.2.4 검증 단계 (Verification Phase)

```
impl Receipt {
pub fn verify(&self, image_id: Digest) -> Result<()> {
match self {
Receipt::Composite(receipt) => {
// STARK 증명 검증
receipt.verify_integrity_with_context(&VerifierContext::default())?;
// Image ID 확인
if receipt.claim.pre.digest() != image_id {
bail!("Image ID mismatch");
}
Ok(())
}
Receipt::Groth16(receipt) => {
// Groth16 증명 검증
receipt.verify_with_context(&VerifierContext::default())?;
Ok(())
}
```

```
}

}

}
```

---

4. 실행 환경 구성

4.1 ExecutorEnv 구조

RISC0의 실행 환경은 `ExecutorEnv` 를 통해 구성됩니다. 이는 Guest 프로그램이 실행될 환경을 정의하는 핵심 구조체입니다.

```
pub  struct ExecutorEnv<'a> {

// 환경 변수 및 인자

env_vars: HashMap<String, String>,

args: Vec<String>,



// 실행 제한

segment_limit_po2: Option<u32>, // 세그먼트 크기 제한 (2의 거듭제곱)

session_limit: Option<u64>, // 세션 사이클 제한

keccak_max_po2: Option<u32>, // Keccak 증명 크기 제한



// 입출력 처리

posix_io: Rc<RefCell<PosixIo<'a>>>, // POSIX 스타일 I/O

slice_io: Rc<RefCell<SliceIoTable<'a>>>, // 슬라이스 기반 I/O
```

```
input: Vec<u8>, // 입력 데이터


// 추적 및 디버깅
trace: Vec<Rc<RefCell<dyn TraceCallback + 'a>>>, // 실행 추적
pprof_out: Option<PathBuf>, // 프로파일링 출력


// 가정 및 증명
assumptions: Rc<RefCell<AssumptionReceipts>>, // 가정 영수증
coprocessor: Option<CoprocessorCallbackRef<'a>>, // 코프로세서 콜백


// 기타
segment_path: Option<SegmentPath>, // 세그먼트 저장 경로
input_digest: Option<Digest>, // 입력 다이제스트
povw_job_id: Option<PovwJobId>, // PoVW 작업 ID
}
```

4.2 ExecutorEnvBuilder 패턴

```
impl<'a> ExecutorEnvBuilder<'a> {
// 기본 설정
pub fn segment_limit_po2(&mut self, limit: u32) -> &mut Self {
// 세그먼트 크기 제한 설정 (메모리 사용량 제어)
// 값을 1 낮추면 메모리 사용량이 약 절반으로 감소
```

```
self.inner.segment_limit_po2 = Some(limit);
self
}

pub fn session_limit(&mut self, limit: Option<u64>) -> &mut Self {
// 전체 세션의 사이클 제한 설정
self.inner.session_limit = limit;
self
}

// 입출력 설정
pub fn write<T: Serialize>(&mut self, data: &T) -> Result<&mut Self> {
// 직렬화된 데이터를 Guest stdin에 작성
Ok(self.write_slice(&to_vec(data)?))
}

pub fn stdin(&mut self, reader: impl Read + 'a) -> &mut Self {
// POSIX 스타일 표준 입력 설정
self.read_fd(fileno::STDIN, BufReader::new(reader))
}

pub fn stdout(&mut self, writer: impl Write + 'a) -> &mut Self {
// POSIX 스타일 표준 출력 설정
```

```
self.write_fd(fileno::STDOUT, writer)
}

// 가정 관리
pub fn add_assumption(&mut self, assumption: impl Into<AssumptionReceipt>) ->
// 컴포지션을 위한 가정 영수증 추가
self.inner.assumptions.borrow_mut().0.push(assumption.into());
self
}

// 추적 및 디버깅
pub fn trace_callback(&mut self, callback: impl TraceCallback + 'a) -> &mut
// 실행 추적을 위한 콜백 등록
self.inner.trace.push(Rc::new(RefCell::new(callback)));
self
}

pub fn enable_profiler<P: AsRef<Path>>(&mut self, path: P) -> &mut Self {
// 성능 프로파일러 활성화
self.inner.pprof_out = Some(path.as_ref().to_path_buf());
self
}
```

```
}
```

4.3 환경 변수 및 설정

```
impl ExecutorEnvBuilder {
pub fn build(&mut self) -> Result<ExecutorEnv<'a>> {
let mut inner = mem::take(&mut self.inner);

// 환경 변수에서 설정 읽기
if inner.pprof_out.is_none() {
if let Ok(env_var) = std::env::var("RISC0_PPROF_OUT") {
inner.pprof_out = Some(env_var.into());
}
}

// Keccak 설정
if let Ok(po2) = std::env::var("RISC0_KECCAK_PO2") {
let po2 = po2.parse::<u32>()?;
if !KECCAK_PO2_RANGE.contains(&(po2 as usize)) {
bail!("invalid keccak po2 {po2}. Expected range: {:?}", KECCAK_PO2_RANGE);
}
inner.keccak_max_po2 = Some(po2);
}
```

```
// 입력 데이터 처리
if !inner.input.is_empty() {
let  reader = Cursor::new(inner.input.clone());
inner.posix_io.borrow_mut().with_read_fd(fileno::STDIN, reader);
}

Ok(inner)
}
}
```

---

5. 실행 프로세스

5.1 전체 실행 흐름

```
|1. 컴파일| -> |2. 환경구성| -> |3. 실행| -> |4. 증명| -> |5. 검증|
|(Method)|     |(Env)| |(Execution)| |(Proving)| |(Verification)|
```

5.2 단계별 상세 분석

5.2.1 컴파일 단계 (Method Creation)

```rust
// 1. Guest 프로그램 작성
// guest/src/main.rs
use risc0_zkvm::guest::env;

fn main() {
// 입력 데이터 읽기
let input: u32 = env::read();

// 계산 수행
let result = fibonacci(input);

// 결과를 Journal에 커밋
env::commit(&result);
}

// 2. 빌드 스크립트에서 ELF 생성
// methods/build.rs
use risc0_build::embed_methods;

fn main() {
// guest 디렉토리의 메서드들을 빌드하고 임베드
embed_methods();
```

```
}

// 3. Image ID 계산
let image_id = compute_image_id(&elf_bytes);
```

5.2.2 환경 구성 단계 (Environment Setup)

```
// Host 프로그램에서 실행 환경 구성
let env = ExecutorEnv::builder()
.write(&input_data)? // 입력 데이터 설정
.segment_limit_po2(20) // 세그먼트 크기 제한 (1M 사이클)
.session_limit(Some(100_000_000)) // 세션 제한 (100M 사이클)
.stdout(&mut stdout_buf) // 표준 출력 설정
.trace_callback(MyTraceCallback::new()) // 추적 콜백 등록
.enable_profiler("profile.pb") // 프로파일러 활성화
.build()?;
```

5.2.3 실행 단계 (Execution Phase)

5.2.3.1 ExecutorImpl 초기화

```
pub struct ExecutorImpl<'a> {
env: ExecutorEnv<'a>, // 실행 환경
image: MemoryImage, // 메모리 이미지
syscall_table: SyscallTable<'a>, // 시스템 콜 테이블
```

```
elf: Option<Vec<u8>>, // ELF 바이너리

profiler: Option<Rc<RefCell<Profiler>>>, // 프로파일러

return_cache: Cell<(u32, u32)>, // 반환값 캐시

}


impl<'a> ExecutorImpl<'a> {

pub fn from_elf(mut env: ExecutorEnv<'a>, elf: &[u8]) -> Result<Self> {

// 1. ELF 바이너리 디코딩

let binary = ProgramBinary::decode(elf)?;

check_program_version(&binary.header)?;


// 2. 메모리 이미지 생성

let image = binary.to_image()?;


// 3. 프로파일러 설정 (필요시)

let profiler = if env.pprof_out.is_some() {

let profiler = Rc::new(RefCell::new(Profiler::new(

&binary,

None,

profiler::read_enable_inline_functions_env_var(),

)?));

env.trace.push(profiler.clone());

Some(profiler)
```

```
} else {
None
};

Self::with_details(env, Some(binary.user_elf), image, profiler)
}
}
```

5.2.3.2 회로 실행기 (Circuit Executor)

```
pub struct Executor<'a, 'b, S: Syscall> {
pc: ByteAddr, // 프로그램 카운터
user_pc: ByteAddr, // 사용자 프로그램 카운터
machine_mode: u32, // 머신 모드
user_cycles: u32, // 사용자 사이클
initial_image: MemoryImage, // 초기 메모리 이미지
pager: PagedMemory, // 페이지 메모리 관리자
terminate_state: Option<TerminateState>, // 종료 상태
read_record: Vec<Vec<u8>>, // 읽기 기록
write_record: Vec<u32>, // 쓰기 기록
syscall_handler: &'a S, // 시스템 콜 핸들러
input_digest: Digest, // 입력 다이제스트
output_digest: Option<Digest>, // 출력 다이제스트
trace: Vec<Rc<RefCell<dyn TraceCallback + 'b>>>, // 추적 콜백
```

```
cycles: SessionCycles, // 세션 사이클 정보
ecall_metrics: EnumMap<EcallKind, EcallMetric>, // Ecall 메트릭
ring: AllocRingBuffer<(ByteAddr, InsnKind, DecodedInstruction)>, // 명령어 링 버퍼
povw_job_id: Option<PovwJobId>, // PoVW 작업 ID
}
```

5.2.3.3 실행 루프

```
impl Executor {
pub fn run(
&mut self,
segment_po2: usize,
max_insn_cycles: usize,
max_cycles: CycleLimit,
callback: impl FnMut(Segment) -> Result<()> + Send,
) -> Result<ExecutorResult> {
let segment_limit: u32 = 1 << segment_po2;
let segment_threshold = segment_limit - max_insn_cycles as u32;
let mut segment_counter = 0u32;

self.reset();

let mut emu = Emulator::new();
```

```
Risc0Machine::resume(self)?;


// 세그먼트 생성을 위한 스레드 생성
let (commit_sender, commit_recv) = sync_channel(MAX_OUTSTANDING_SEGMENTS - 1);
let  initial_image = self.initial_image.clone();


let (initial_digest, post_digest, post_image) = thread::scope(|scope| {
let  segment_callback_thread = scope.spawn(move ||
create_segments(initial_image, commit_recv, callback)
);


// 메인 실행 루프
while  self.terminate_state.is_none() {
// 사이클 제한 확인
match  max_cycles {
CycleLimit::Hard(max_cycles) => {
if  self.cycles.user >= max_cycles {
bail!("Session limit exceeded: {} >= {max_cycles}", self.cycles.user);
}
}
CycleLimit::Soft(max_cycles) => {
if  self.cycles.user >= max_cycles {
break;
```

```
}
}
CycleLimit::None => {}
}

// 세그먼트 분할 확인
if self.segment_cycles() > segment_threshold {
tracing::debug!(
"split(phys: {} + pager: {} + reserved: {RESERVED_CYCLES}) = {} >= {segment_thre
self.user_cycles, self.pager.cycles, self.segment_cycles()
);

// 세그먼트 생성 요청
Risc0Machine::suspend(self)?;
let partial_image = self.pager.commit();

let req = CreateSegmentRequest {
partial_image,
page_indexes: self.pager.page_indexes(),
input_digest: self.input_digest,
output_digest: self.output_digest,
read_record: std::mem::take(&mut self.read_record),
write_record: std::mem::take(&mut self.write_record),
```

```
user_cycles: self.user_cycles,

pager_cycles: self.pager.cycles,

terminate_state: self.terminate_state,

segment_threshold,

po2: segment_po2  as u32,

index: segment_counter  as u64,

dump_path: None,

povw_nonce: self.povw_nonce(segment_counter),

};



if  commit_sender.send(req).is_err() {

return Err(segment_callback_thread.join().unwrap().unwrap_err());

}



segment_counter = segment_counter.checked_add(1)

.context("segment_counter overflow")?;



// 사이클 카운터 업데이트

let  total_cycles = 1 << segment_po2;

let  pager_cycles = self.pager.cycles as u64;

let  user_cycles = self.user_cycles as u64;

self.cycles.total += total_cycles;

self.cycles.paging += pager_cycles;
```

```
self.cycles.reserved += total_cycles - pager_cycles - user_cycles;
self.user_cycles = 0;
self.pager.reset();

Risc0Machine::resume(self)?;
}

// 명령어 실행
let result = Risc0Machine::step(&mut emu, self);
if let Err(err) = result {
self.dump();
return Err(err);
}
}

// 최종 세그먼트 처리
Risc0Machine::suspend(self)?;
let final_cycles = self.segment_cycles().next_power_of_two();
let final_po2 = log2_ceil(final_cycles as usize);
let partial_image = self.pager.commit();

let req = CreateSegmentRequest {
partial_image,
```

```
page_indexes: self.pager.page_indexes(),
input_digest: self.input_digest,
output_digest: self.output_digest,
read_record: std::mem::take(&mut  self.read_record),
write_record: std::mem::take(&mut  self.write_record),
user_cycles: self.user_cycles,
pager_cycles: self.pager.cycles,
terminate_state: self.terminate_state,
segment_threshold: 0,
po2: final_po2  as u32,
index: segment_counter  as u64,
dump_path: None,
povw_nonce: self.povw_nonce(segment_counter),
};


if  commit_sender.send(req).is_err() {
return Err(segment_callback_thread.join().unwrap().unwrap_err());
}


let  final_cycles = final_cycles  as u64;
let  user_cycles = self.user_cycles as u64;
let  pager_cycles = self.pager.cycles as u64;
self.cycles.total += final_cycles;
```

```
self.cycles.paging += pager_cycles;

self.cycles.reserved += final_cycles - pager_cycles - user_cycles;



drop(commit_sender);

segment_callback_thread.join().unwrap()

})?;



// ExecutorResult 생성

let  session_claim = Rv32imV2Claim {

pre_state: initial_digest,

post_state: post_digest,

input: self.input_digest,

output: self.output_digest,

terminate_state: self.terminate_state,

shutdown_cycle: None,

};



Ok(ExecutorResult {

segments: segment_counter  as u64 + 1,

post_image,

user_cycles: self.cycles.user,

total_cycles: self.cycles.total,

paging_cycles: self.cycles.paging,
```

```
reserved_cycles: self.cycles.reserved,

claim: session_claim,

})

}

}
```

---

6. 세그먼트 관리

6.1 세그먼트 구조

세그먼트는 RISC0에서 실행을 분할하는 기본 단위입니다. 메모리 사용량을 제한하고 병렬 처리를 가능하게 합니다.

```
#[derive(Clone, Serialize, Deserialize)]

pub struct Segment {

/// 세그먼트 인덱스

pub index: u32,


/// 내부 회로 세그먼트

pub(crate) inner: risc0_circuit_rv32im::execute::Segment,


/// 출력 (마지막 세그먼트에만 존재)

pub(crate) output: Option<Output>,

}
```

```rust
// 회로 레벨 세그먼트 구조
pub struct Segment {
/// 초기 희소 메모리 상태
pub partial_image: MemoryImage,

/// 세그먼트 클레임
pub claim: Rv32imV2Claim,

/// 호스트->게스트 I/O 기록
pub read_record: Vec<Vec<u8>>,

/// 게스트->호스트 I/O 기록
pub write_record: Vec<u32>,

/// 중단 사이클
pub suspend_cycle: u32,

/// 페이징 사이클
pub paging_cycles: u32,

/// 세그먼트 임계값
pub segment_threshold: u32,
```

```rust
/// 2의 거듭제곱 크기
pub po2: u32,

/// 세그먼트 인덱스
pub index: u64,

/// PoVW 논스 (선택적)
pub povw_nonce: Option<PovwNonce>,
}
```

6.2 세그먼트 생성 프로세스

```rust
struct CreateSegmentRequest {
partial_image: WorkingImage,
page_indexes: BTreeSet<u32>,
input_digest: Digest,
output_digest: Option<Digest>,
read_record: Vec<Vec<u8>>,
write_record: Vec<u32>,
user_cycles: u32,
pager_cycles: u32,
terminate_state: Option<TerminateState>,
```

```
segment_threshold: u32,

po2: u32,

index: u64,

povw_nonce: Option<PovwNonce>,

dump_path: Option<std::ffi::OsString>,

}


fn create_segments(

initial_image: MemoryImage,

recv: std::sync::mpsc::Receiver<CreateSegmentRequest>,

mut callback: impl FnMut(Segment) -> Result<()>,

) -> Result<(Digest, Digest, MemoryImage)> {

let mut existing_image = initial_image;

let initial_digest = existing_image.image_id();


while let Ok(req) = recv.recv() {

// 이전 상태 다이제스트 계산

let pre_digest = existing_image.image_id();


// 부분 이미지 계산

let partial_image = compute_partial_image(&mut existing_image, req.page_indexe


// 메모리 이미지 업데이트
```

```
for (idx, page) in  req.partial_image.pages {
existing_image.set_page(idx, page);
}
existing_image.update_digests();
let  post_digest = existing_image.image_id();


// 세그먼트 생성
let  segment = Segment {
partial_image,
claim: Rv32imV2Claim {
pre_state: pre_digest,
post_state: post_digest,
input: req.input_digest,
output: req.output_digest,
terminate_state: req.terminate_state,
shutdown_cycle: None,
},
read_record: req.read_record,
write_record: req.write_record,
suspend_cycle: req.user_cycles,
paging_cycles: req.pager_cycles,
po2: req.po2,
index: req.index,
```

```
segment_threshold: req.segment_threshold,
povw_nonce: req.povw_nonce,
};

// 디버그 덤프 (필요시)
if let Some(dump_path) = req.dump_path {
let bytes = segment.encode()?;
std::fs::write(dump_path, bytes)?;
break;
}

callback(segment)?;
}

Ok((initial_digest, existing_image.image_id(), existing_image))
}
```

6.3 세그먼트 참조 시스템

RISC0는 다양한 세그먼트 저장 방식을 지원합니다:

```
/// 세그먼트 참조 트레이트
pub trait SegmentRef: Send {
fn resolve(&self) -> Result<Segment>;
```

```rust
}

/// 메모리 내 저장
#[derive(Clone, Serialize, Deserialize)]
pub struct SimpleSegmentRef {
segment: Segment,
}

impl SegmentRef for SimpleSegmentRef {
fn resolve(&self) -> Result<Segment> {
Ok(self.segment.clone())
}
}

/// 파일 기반 저장
pub struct FileSegmentRef {
path: PathBuf,
_dir: SegmentPath,
}

impl SegmentRef for FileSegmentRef {
fn resolve(&self) -> Result<Segment> {
let contents = fs::read(&self.path)?;
```

```
let segment = bincode::deserialize(&contents)?;
Ok(segment)
}
}

impl FileSegmentRef {
pub fn new(segment: &Segment, dir: &SegmentPath) -> Result<Self> {
let path = dir.path().join(format!("{}.bincode", segment.index));
fs::write(&path, bincode::serialize(&segment)?)?;
Ok(Self {
path,
_dir: dir.clone(),
})
}
}

/// Null 참조 (개발 모드용)
#[derive(Serialize, Deserialize)]
pub struct NullSegmentRef;

impl SegmentRef for NullSegmentRef {
fn resolve(&self) -> anyhow::Result<Segment> {
unimplemented!()
```

```
}

}
```

6.4 세션 구조

세션은 전체 프로그램 실행을 나타내며 여러 세그먼트로 구성됩니다:

```
pub struct Session {

/// 구성 세그먼트들

pub segments: Vec<Box<dyn SegmentRef>>,


/// 입력 다이제스트

pub input: Digest,


/// 공개 커밋된 데이터

pub journal: Option<Journal>,


/// 종료 코드

pub exit_code: ExitCode,


/// 가정들 (호스트가 해결)

pub assumptions: Vec<(Assumption, AssumptionReceipt)>,


/// MMR 가정들
```

```rust
pub  mmr_assumptions: Vec<AssumptionReceipt>,


/// 이벤트 훅들
pub  hooks: Vec<Box<dyn SessionEvents>>,


/// 사이클 정보
pub  user_cycles: u64,
pub  paging_cycles: u64,
pub  reserved_cycles: u64,
pub  total_cycles: u64,


/// 시스템 상태
pub  pre_state: SystemState,
pub  post_state: SystemState,


/// 대기 중인 Keccak 증명 요청들
pub(crate) pending_keccaks: Vec<ProveKeccakRequest>,


/// 메트릭
pub(crate) ecall_metrics: EnumMap<EcallKind, EcallMetric>,
pub(crate) syscall_metrics: EnumMap<SyscallKind, SyscallMetric>,


/// PoVW 작업 ID
```

```
pub(crate) povw_job_id: Option<PovwJobId>,


/// 실행 시간
pub(crate) execution_time: Duration,
}
```

6.5 세그먼트 실행 및 재현

```
impl Segment {
/// 세그먼트 재실행 (디버깅/검증용)
pub fn execute(&self) -> Result<()> {
let handler = SegmentSyscallHandler {
segment: self,
read_pos: Cell::new(0),
write_pos: Cell::new(0),
};


let max_insn_cycles = if self.po2 >= 15 {
MAX_INSN_CYCLES
} else {
MAX_INSN_CYCLES_LOWER_PO2
};
```

```
Executor::new(
self.partial_image.clone(),
&handler,
None,
vec![],
None
).run(
self.po2 as usize,
max_insn_cycles,
CycleLimit::Soft(self.suspend_cycle.into()),
|_| Ok(()),
)?;


Ok(())
}


/// 세그먼트 직렬화
pub fn encode(&self) -> Result<Vec<u8>> {
Ok(postcard::to_stdvec(self)?)
}


/// 세그먼트 역직렬화
pub fn decode(bytes: &[u8]) -> Result<Self> {
```

```
Ok(postcard::from_bytes(bytes)?)

}

}


/// 세그먼트 재실행을 위한 시스템 콜 핸들러

struct SegmentSyscallHandler<'a> {

segment: &'a Segment,

read_pos: Cell<usize>,

write_pos: Cell<usize>,

}


impl Syscall for SegmentSyscallHandler<'_> {

fn  host_read(&self, _ctx: &mut  dyn SyscallContext, _fd: u32, buf: &mut [u8]) -

let  pos = self.read_pos.replace(self.read_pos.get() + 1);

let  read_record = &self.segment.read_record[pos];

buf.copy_from_slice(read_record);

Ok(read_record.len() as u32)

}


fn  host_write(&self, _ctx: &mut  dyn SyscallContext, _fd: u32, _buf: &[u8]) ->

let  pos = self.write_pos.replace(self.write_pos.get() + 1);

Ok(self.segment.write_record[pos])

}
```

```
}
```

---

7. 증명 시스템

7.1 ProverServer 아키텍처

RISC0의 증명 시스템은 `ProverServer` 트레이트를 중심으로 구성됩니다:

```
pub trait ProverServer {
/// ELF 바이너리 증명
fn prove_with_ctx(
&self,
env: ExecutorEnv<'_>,
ctx: &VerifierContext,
elf: &[u8],
) -> Result<ProveInfo>;

/// 세션 증명
fn prove_session(&self, ctx: &VerifierContext, session: &Session) -> Result<Pro

/// 개별 세그먼트 증명
fn prove_segment(&self, ctx: &VerifierContext, segment: &Segment) -> Result<Seg
```

```
/// 세그먼트 사전 검사
fn  segment_preflight(&self, segment: &Segment) -> Result<PreflightResults>;


/// 사전 검사된 세그먼트 증명
fn  prove_segment_core(
&self,
ctx: &VerifierContext,
preflight_results: PreflightResults,
) -> Result<SegmentReceipt>;


/// Keccak 증명
fn  prove_keccak(&self, request: &ProveKeccakRequest) -> Result<SuccinctReceipt<


/// 재귀 증명 연산들
fn  lift(&self, receipt: &SegmentReceipt) -> Result<SuccinctReceipt<ReceiptClaim
fn  join(&self, a: &SuccinctReceipt<ReceiptClaim>, b: &SuccinctReceipt<ReceiptCl
fn  union(&self, a: &SuccinctReceipt<Unknown>, b: &SuccinctReceipt<Unknown>) ->
fn  resolve(&self, conditional: &SuccinctReceipt<ReceiptClaim>, assumption: &Suc


/// PoVW (Proof of Verifiable Work) 연산들
fn  lift_povw(&self, receipt: &SegmentReceipt) -> Result<SuccinctReceipt<WorkCla
fn  join_povw(&self, a: &SuccinctReceipt<WorkClaim<ReceiptClaim>>, b: &SuccinctR
fn  unwrap_povw(&self, a: &SuccinctReceipt<WorkClaim<ReceiptClaim>>) -> Result<S
```

```
/// 증명 압축
fn composite_to_succinct(&self, receipt: &CompositeReceipt) -> Result<SuccinctR
fn succinct_to_groth16(&self, receipt: &SuccinctReceipt<ReceiptClaim>) -> Resul
fn identity_p254(&self, a: &SuccinctReceipt<ReceiptClaim>) -> Result<SuccinctRe
}
```

7.2 3계층 재귀 증명 구조

7.2.1 Layer 1: Segment Proofs (세그먼트 증명)

```
// 세그먼트 증명 생성
impl ProverServer for ProverImpl {
fn prove_segment(&self, ctx: &VerifierContext, segment: &Segment) -> Result<Seg
// 1. 사전 검사 실행
let preflight_results = self.segment_preflight(segment)?;

// 2. 핵심 증명 생성
self.prove_segment_core(ctx, preflight_results)
}

fn prove_segment_core(
&self,
ctx: &VerifierContext,
```

```
preflight_results: PreflightResults,

) -> Result<SegmentReceipt> {

// RV32IM 회로에서 STARK 증명 생성

let  hal_pair = self.get_hal_pair()?;

let  prover = SegmentProver::new(&hal_pair);



// 실행 추적을 기반으로 증명 생성

let  proof = prover.prove_segment(&preflight_results)?;



// 세그먼트 영수증 생성

Ok(SegmentReceipt {

seal: proof.seal,

claim: preflight_results.claim,

verifier_parameters: SegmentReceiptVerifierParameters::default().digest(),

})

}

}
```

특징:

- **목적**: 개별 실행 세그먼트에 대한 증명
- **프로토콜**: zk-STARK
- **크기**: 각 세그먼트는 최대 2^20 사이클 (약 1M 사이클)
- **내용**: RV32IM 명령어 실행, 메모리 일관성, 시스템 콜 처리

7.2.2 Layer 2: Recursion Proofs (재귀 증명)

재귀 증명은 여러 세그먼트 증명을 하나로 결합하는 역할을 합니다:

```
/// Lift: 세그먼트 증명을 재귀 증명으로 변환
pub fn lift(segment_receipt: &SegmentReceipt) -> Result<SuccinctReceipt<Receip
tracing::debug!("Proving lift: claim = {:#?}", segment_receipt.claim);

// Lift 프로그램으로 프로버 생성
let mut prover = Prover::new_lift(segment_receipt, ProverOpts::succinct())?;

// 재귀 회로에서 세그먼트 증명 검증
let receipt = prover.prover.run()?;
let claim_decoded = ReceiptClaim::decode(&mut receipt.out_stream())?;

// 클레임 병합 및 검증
let claim = claim_decoded.merge(&segment_receipt.claim)?;

make_succinct_receipt(prover, receipt, claim)
}

/// Join: 두 재귀 증명을 하나로 결합
pub fn join(
a: &SuccinctReceipt<ReceiptClaim>,
```

```
b: &SuccinctReceipt<ReceiptClaim>,
) -> Result<SuccinctReceipt<ReceiptClaim>> {
tracing::debug!("Proving join: a.claim = {:#?}", a.claim);
tracing::debug!("Proving join: b.claim = {:#?}", b.claim);

// Join 프로그램으로 프로버 생성
let mut prover = Prover::new_join(a, b, ProverOpts::succinct())?;
let receipt = prover.prover.run()?;

let claim_decoded = ReceiptClaim::decode(&mut receipt.out_stream())?;

// 예상 클레임 계산 및 병합
let claim = claim_decoded.merge(&a.claim.join(&b.claim)?.value()?)?;

make_succinct_receipt(prover, receipt, claim)
}

/// Resolve: 조건부 증명에서 가정 해결
pub fn resolve<Claim>(
conditional: &SuccinctReceipt<ReceiptClaim>,
assumption: &SuccinctReceipt<Claim>,
) -> Result<SuccinctReceipt<ReceiptClaim>>
where
```

```
Claim: risc0_binfmt::Digestible + Debug,
{
tracing::debug!("Proving resolve: conditional.claim = {:#?}", conditional.claim)
tracing::debug!("Proving resolve: assumption.claim = {:#?}", assumption.claim);

// Resolve 프로그램으로 프로버 생성
let mut prover = Prover::new_resolve(conditional, assumption, ProverOpts::succ
let receipt = prover.prover.run()?;
let claim_decoded = ReceiptClaim::decode(&mut receipt.out_stream())?;

// 해결된 클레임 계산
let claim = conditional
.claim
.as_value()
.context("conditional receipt claim is pruned")?
.resolve(&assumption.claim)
.context("failed to compute resolved claim")?
.merge(&claim_decoded)
.context("failed to merge resolved and decoded claims")?;

make_succinct_receipt(prover, receipt, claim)
}
```

특징:

- **목적**: 다중 세그먼트 증명을 하나로 결합
- **프로토콜**: zk-STARK (재귀 회로)
- **연산**: Lift, Join, Resolve, Union
- **확장성**: 임의 개수의 세그먼트 지원

7.2.3 Layer 3: Groth16 Proofs (최종 압축)

```
/// Identity P254: Poseidon254 해시 함수로 증명 변환
pub fn identity_p254(
inner: &SuccinctReceipt<ReceiptClaim>,
) -> Result<SuccinctReceipt<ReceiptClaim>> {
tracing::debug!("identity_p254");

// Poseidon254 해시 함수 사용 설정
let opts = ProverOpts::succinct()
.with_hashfn("poseidon_254".to_string())
.with_control_ids(vec![BN254_IDENTITY_CONTROL_ID]);

let mut prover = Prover::new_identity(inner, opts)?;
let receipt = prover.prover.run()?;
let claim = MaybePruned::Value(ReceiptClaim::decode(&mut receipt.out_stream())
.merge(&inner.claim)?;
```

```
// 제어 ID 포함 증명 생성
let  hashfn = prover.opts.hash_suite()?.hashfn;
let  control_inclusion_proof = prover.control_inclusion_proof()?;
let  control_root = control_inclusion_proof.root(&prover.control_id, hashfn.as_r


let  params = SuccinctReceiptVerifierParameters {
control_root,
inner_control_root: Some(inner.control_root()?),
proof_system_info: PROOF_SYSTEM_INFO,
circuit_info: CircuitImpl::CIRCUIT_INFO,
};


Ok(SuccinctReceipt {
seal: receipt.seal,
hashfn: prover.opts.hashfn.clone(),
control_id: prover.control_id,
control_inclusion_proof,
claim,
verifier_parameters: params.digest(),
})
}


/// Groth16 증명 생성
```

```rust
impl ProverServer for ProverImpl {
fn succinct_to_groth16(
&self,
receipt: &SuccinctReceipt<ReceiptClaim>,
) -> Result<Groth16Receipt<ReceiptClaim>> {
// 1. Poseidon254로 변환
let ident_receipt = self.identity_p254(receipt)?;


// 2. Groth16 압축
let seal_bytes = ident_receipt.get_seal_bytes();
let seal = shrink_wrap(&seal_bytes)?.to_vec();


Ok(Groth16Receipt {
seal,
claim: receipt.claim.clone(),
verifier_parameters: Groth16ReceiptVerifierParameters::default().digest(),
})
}
}
```

특징:

- **목적**: 최종 증명 크기 최소화
- **프로토콜**: Groth16 zk-SNARK

- **크기**: 고정 크기 증명 (약 200바이트)
- **검증**: 빠른 검증 시간 (밀리초 단위)
- **제약**: 신뢰할 수 있는 설정 필요

7.3 Proof of Verifiable Work (PoVW)

PoVW는 계산 작업의 양을 증명하는 메커니즘입니다:

```
/// PoVW를 포함한 Lift
pub fn lift_povw(
segment_receipt: &SegmentReceipt,
) -> Result<SuccinctReceipt<WorkClaim<ReceiptClaim>>> {
tracing::debug!("Proving lift_povw: claim = {:#?}", segment_receipt.claim);


let mut prover = Prover::new_lift_povw(segment_receipt, ProverOpts::succinct()
let receipt = prover.prover.run()?;


let mut out_stream = receipt.out_stream();
let claim_decoded = WorkClaim::<ReceiptClaim>::decode_from_seal(&mut out_strea


// 작업 클레임에 세그먼트 클레임 병합
let mut claim = claim_decoded;
claim.claim.merge_with(&segment_receipt.claim.clone().into())
.context("failed to merge segment receipt claim into decode claim")?;
```

```
make_succinct_receipt(prover, receipt, claim)
}

/// PoVW Join
pub fn join_povw(
a: &SuccinctReceipt<WorkClaim<ReceiptClaim>>,
b: &SuccinctReceipt<WorkClaim<ReceiptClaim>>,
) -> Result<SuccinctReceipt<WorkClaim<ReceiptClaim>>> {
tracing::debug!("Proving join_povw: a.claim = {:#?}", a.claim);
tracing::debug!("Proving join_povw: b.claim = {:#?}", b.claim);

let mut prover = Prover::new_join_povw(a, b, false, ProverOpts::succinct())?;
let receipt = prover.prover.run()?;

let claim_decoded = WorkClaim::<ReceiptClaim>::decode_from_seal(&mut receipt.o

// 작업 값 결합 및 논스 범위 검증
let claim = claim_decoded.merge(&a.claim.join(&b.claim)?.value()?)?;

make_succinct_receipt(prover, receipt, claim)
}

/// PoVW Unwrap: 작업 클레임에서 일반 클레임으로 변환
```

```
pub fn unwrap_povw(
a: &SuccinctReceipt<WorkClaim<ReceiptClaim>>,
) -> Result<SuccinctReceipt<ReceiptClaim>> {
tracing::debug!("Proving unwrap_povw: a.claim = {:#?}", a.claim);


let mut prover = Prover::new_unwrap_povw(a, ProverOpts::succinct())?;
let receipt = prover.prover.run()?;


let claim_decoded = ReceiptClaim::decode(&mut receipt.out_stream())?;
let claim = MaybePruned::Value(claim_decoded).merge(&a.claim.as_value()?.claim)


make_succinct_receipt(prover, receipt, claim)
}
```

7.4 컴포지트 영수증 압축

```
impl<P> Compress<Claim> for P
where
P: Lift<Claim> + Join<Claim> + Resolve<Claim> + ?Sized,
{
fn composite_to_succinct(
&self,
```

```
composite_receipt: &CompositeReceipt,

) -> anyhow::Result<SuccinctReceipt<Claim>> {

// 1. 모든 세그먼트 영수증을 하나의 간결한 영수증으로 압축

let  continuation_receipt = composite_receipt

.segments

.iter()

.try_fold(

None,

|left: Option<SuccinctReceipt<Claim>>, right: &SegmentReceipt| -> Result<_> {

Ok(Some(match  left {

Some(left) => self.join(&left, &self.lift(right)?)?,

None => self.lift(right)?,

}))

},

)?

.ok_or_else(|| anyhow!("malformed composite receipt has no continuation segment


// 2. 가정들을 압축하고 해결하여 최종 간결한 영수증 생성

composite_receipt.assumption_receipts.iter().try_fold(

continuation_receipt,

|conditional: SuccinctReceipt<Claim>, assumption: &InnerAssumptionReceipt| match

InnerAssumptionReceipt::Succinct(assumption) => {

self.resolve(&conditional, assumption)
```

```
}
InnerAssumptionReceipt::Composite(assumption) => {
let  assumption_succinct = self.composite_to_succinct(assumption)?;
self.resolve(&conditional, &SuccinctReceipt::<ReceiptClaim>::into_unknown(assump
}
InnerAssumptionReceipt::Fake(_) => {
bail!("compressing composite receipts with fake receipt assumptions is not suppo
}
InnerAssumptionReceipt::Groth16(_) => {
bail!("compressing composite receipts with Groth16 receipt assumptions is not su
}
},
)
}
}
```

7.5 보안 매개변수

5.1.1 Layer 1: Segment Proofs

- **목적**: 개별 실행 세그먼트에 대한 증명
- **프로토콜**: zk-STARK
- **특징**:
- 각 세그먼트는 최대 2^20 사이클

- RV32IM 명령어 실행 증명
- 메모리 일관성 증명

5.1.2 Layer 2: Recursion Proofs

- **목적**: 다중 세그먼트 증명을 하나로 결합
- **프로토콜**: zk-STARK (재귀 회로)
- **특징**:
- 증명 크기 압축
- 임의 개수의 세그먼트 지원
- 병렬 처리 가능

5.1.3 Layer 3: Groth16 Proofs

- **목적**: 최종 증명 크기 최소화
- **프로토콜**: Groth16 zk-SNARK
- **특징**:
- 고정 크기 증명 (약 200바이트)
- 빠른 검증 시간
- 신뢰할 수 있는 설정 필요

5.2 보안 매개변수

```
// risc0-zkp/src/lib.rs에서 정의된 보안 매개변수
pub const QUERIES: usize = 50; // FRI 쿼리 수 (97비트 보안)
pub const ZK_CYCLES: usize = 1024; // 영지식성을 위한 사이클
pub const INV_RATE: usize = 4; // Reed-Solomon 확장 비율의 역수
```

```
pub const FRI_FOLD: usize = 16; // FRI 폴딩 팩터
```

---

6. 회로 구현

6.1 RV32IM 회로 구조

6.1.1 명령어 처리

```
// risc0-circuit-rv32im에서 구현된 명령어 처리
pub struct RV32IMCircuit {
// 레지스터 상태
registers: [u32; 32],
// 메모리 상태
memory: MemoryState,
// 프로그램 카운터
pc: u32,
// 실행 추적
trace: ExecutionTrace,
}

impl RV32IMCircuit {
// 명령어 실행
pub fn execute_instruction(&mut self, instruction: u32) -> Result<()> {
match decode_instruction(instruction) {
```

```
Instruction::Add { rd, rs1, rs2 } => {
self.registers[rd] = self.registers[rs1]
.wrapping_add(self.registers[rs2]);
}
Instruction::Load { rd, rs1, imm } => {
let  addr = self.registers[rs1].wrapping_add(imm  as u32);
self.registers[rd] = self.memory.load(addr)?;
}
// ... 기타 명령어들
}
self.pc += 4;
Ok(())
}
}
```

6.1.2 제약 조건 시스템

```
// 산술 제약 조건 예시
pub  fn  add_constraints(builder: &mut ConstraintBuilder) {
// 덧셈 명령어 제약 조건
builder.when(is_add_instruction).assert_eq(
next_register_value,
register_a + register_b
```

```
);

// 메모리 일관성 제약 조건
builder.when(is_memory_access).assert_eq(
memory_value,
lookup_memory_table(address)
);

// 프로그램 카운터 제약 조건
builder.when(is_normal_instruction).assert_eq(
next_pc,
current_pc + 4
);
}
```

6.2 재귀 회로 구조

6.2.1 증명 검증 회로

```
// 재귀 회로에서 이전 증명을 검증
pub struct RecursionCircuit {
// 이전 증명들
prev_proofs: Vec<Proof>,
// 공개 입력
```

```
public_inputs: Vec<FieldElement>,

// 검증 키

verification_key: VerificationKey,

}



impl RecursionCircuit {

pub  fn  verify_proofs(&self) -> Result<bool> {

for  proof  in &self.prev_proofs {

// FRI 검증

if !verify_fri_proof(proof, &self.verification_key)? {

return Ok(false);

}



// 제약 조건 검증

if !verify_constraints(proof)? {

return Ok(false);

}

}

Ok(true)

}

}
```

7. 호스트-게스트 통신

7.1 시스템 콜 메커니즘

7.1.1 시스템 콜 테이블

```
// 시스템 콜 등록 및 처리
pub struct SyscallTable<'a> {
syscalls: HashMap<String, Box<dyn SyscallHandler + 'a>>,
assumptions_used: Arc<Mutex<HashMap<Assumption, u64>>>,
mmr_assumptions: RefCell<Option<Vec<MmrAssumption>>>,
}

impl<'a> SyscallTable<'a> {
pub fn from_env(env: &ExecutorEnv<'a>) -> Self {
let mut table = Self::new();

// 기본 시스템 콜들 등록
table.register("sys_read", Box::new(SysRead::new(&env.stdin)));
table.register("sys_write", Box::new(SysWrite::new(&env.stdout)));
table.register("sys_random", Box::new(SysRandom::new()));

// 사용자 정의 시스템 콜들 등록
for (name, handler) in &env.syscalls {
```

```
table.register(name, handler.clone());
}

table
}
}
```

7.1.2 시스템 콜 실행

```
// ExecutorImpl에서 시스템 콜 처리
impl CircuitSyscall for ExecutorImpl<'_> {
fn host_read(
&self,
ctx: &mut dyn CircuitSyscallContext,
fd: u32,
buf: &mut [u8],
) -> Result<u32> {
// 1. 시스템 콜 이름 읽기
let name_ptr = ByteAddr(fd);
let syscall = ctx.peek_string(name_ptr)?;

// 2. 해당 핸들러 찾기
let handler = self.syscall_table
.get_syscall(&syscall)
```

```
.context(format!("Unknown syscall: {syscall:?}"))?;


// 3. 시스템 콜 실행
let mut to_guest = vec![0u32; words];
let result = handler
.borrow_mut()
.syscall(&syscall, &mut ctx, &mut to_guest)?;


// 4. 결과를 게스트에게 반환
let bytes = bytemuck::cast_slice(to_guest.as_slice());
buf.copy_from_slice(&bytes[..buf.len()]);


Ok(buf.len() as u32)
}
}
```

7.2 입출력 처리

7.2.1 Journal 작성

```
// Guest 코드에서 Journal에 데이터 작성
use risc0_zkvm::guest::env;


fn main() {
```

```rust
let input: u32 = env::read();
let result = compute_something(input);

// Journal에 결과 커밋
env::commit(&result);
}
```

7.2.2 Host에서 Journal 읽기

```rust
// Host 코드에서 Journal 읽기
let receipt = prover.prove(elf, &input)?;
let journal = receipt.journal;

// Journal에서 결과 추출
let result: u32 = journal.decode()?;
println!("Computation result: {}", result);
```

---

8. 보안 모델

8.1 암호학적 가정

8.1.1 해시 함수 보안

- **사용 함수**: BLAKE2b, SHA-256

- **가정**: 충돌 저항성, 일방향성
- **용도**: Merkle tree, 커밋먼트

8.1.2 유한체 연산

- **체**: Goldilocks field (p = 2^64 - 2^32 + 1)
- **특징**: 64비트 연산에 최적화
- **보안**: 이산 로그 문제의 어려움

8.1.3 FRI 프로토콜

- **가정**: Reed-Solomon 코드의 거리 특성
- **매개변수**: 50 쿼리로 97비트 보안 달성
- **검증**: 다항식 저차수 테스트

8.2 사운드니스 분석

```
// 사운드니스 계산기 (soundness.rs)
pub fn calculate_soundness(
code_rate: f64,
num_queries: usize,
fri_fold_factor: usize,
) -> f64 {
// FRI 사운드니스 계산
let fri_soundness = (1.0 - code_rate).powi(num_queries as i32);

// 전체 시스템 사운드니스
```

```
let  system_soundness = fri_soundness *
calculate_constraint_soundness() *
calculate_permutation_soundness();


-system_soundness.log2()
}
```

---

9. 성능 최적화

9.1 GPU 가속

9.1.1 CUDA 지원

```
// CUDA 기능 활성화
[features]
cuda = [
"prove",
"risc0-circuit-rv32im/cuda",
"risc0-circuit-recursion/cuda",
"risc0-zkp/cuda",
]
```

9.1.2 Metal 지원 (macOS)

```rust
// Metal 가속 (Apple Silicon에서 기본 활성화)
#[cfg(any(target_os="macos", target_os="ios"))]
use metal::*;

impl MetalProver {
pub fn prove_segment(&self, segment: &Segment) -> Result<Proof> {
// Metal 컴퓨트 셰이더로 증명 생성
let command_buffer = self.command_queue.new_command_buffer();
let compute_encoder = command_buffer.new_compute_command_encoder();

// 병렬 처리로 제약 조건 검증
compute_encoder.set_compute_pipeline_state(&self.constraint_pipeline);
compute_encoder.dispatch_threads(thread_count, threads_per_group);

compute_encoder.end_encoding();
command_buffer.commit();
command_buffer.wait_until_completed();

Ok(proof)
}
}
```

9.2 메모리 최적화

9.2.1 세그먼트 기반 실행

```
// 메모리 사용량을 제한하기 위한 세그먼트 분할
pub const DEFAULT_SEGMENT_LIMIT_PO2: usize = 20; // 2^20 사이클
pub const MAX_CYCLES_PO2: usize = 24; // 2^24 사이클

impl Executor {
pub fn run_segment(&mut self, limit: usize) -> Result<SegmentResult> {
let mut cycles = 0;

while cycles < limit && !self.is_terminated() {
self.step()?;
cycles += 1;

// 메모리 압박 시 조기 종료
if self.memory_usage() > MAX_MEMORY_USAGE {
break;
}
}

Ok(SegmentResult {
cycles,
final_state: self.get_state(),
```

```
trace: self.take_trace(),
})
}
}
```

9.3 병렬 처리

9.3.1 세그먼트 병렬 증명

```
use rayon::prelude::*;

pub fn prove_session_parallel(session: &Session) -> Result<Receipt> {
// 세그먼트들을 병렬로 증명
let segment_receipts: Result<Vec<_>> = session.segments
.par_iter()
.map(|segment| prove_segment(segment))
.collect();

let receipts = segment_receipts?;

// 재귀적으로 결합
prove_recursion_tree(receipts)
}
```

10. 실제 구현 세부사항

10.1 프로젝트 구조 분석

10.1.1 Cargo Workspace 구성

```
# 루트 Cargo.toml
[workspace]
members = [
"risc0/zkvm", # 메인 zkVM 인터페이스
"risc0/zkp", # 영지식 증명 코어
"risc0/circuit/rv32im", # RISC-V 회로
"risc0/circuit/recursion", # 재귀 회로
"risc0/core", # 공통 타입 및 유틸리티
"risc0/groth16", # Groth16 구현
"risc0/build", # 빌드 도구
"risc0/cargo-risczero", # Cargo 플러그인
]
```

10.1.2 의존성 관계

```
risc0-zkvm
├── risc0-zkp (증명 시스템)
├── risc0-circuit-rv32im (RV32IM 회로)
├── risc0-circuit-recursion (재귀 회로)
```

```
├── risc0-groth16 (Groth16 구현)
└── risc0-core (공통 컴포넌트)
```

10.2 빌드 시스템

10.2.1 Guest 프로그램 빌드

```rust
// build.rs에서 guest 프로그램 빌드
use risc0_build::embed_methods;

fn main() {
// guest 디렉토리의 메서드들을 빌드하고 임베드
embed_methods();
}
```

10.2.2 크로스 컴파일 설정

```toml
# .cargo/config.toml
[target.riscv32im-risc0-zkvm-elf]
runner = "r0vm"

[build]
target = "riscv32im-risc0-zkvm-elf"
```

10.3 개발 도구

10.3.1 cargo-risczero

```
# 새 프로젝트 생성
cargo risczero new my_project

# 프로젝트 구조
my_project/
├── Cargo.toml
├── host/
| └── src/main.rs # Host 프로그램
├── methods/
| ├── guest/src/main.rs # Guest 프로그램
| └── build.rs # 빌드 스크립트
└── core/
└── src/lib.rs # 공통 코드
```

10.3.2 디버깅 지원

```
// GDB 디버깅 지원
impl ExecutorImpl {
pub fn run_with_debugger(&mut self) -> Result<()> {
let debugger = GdbExecutor::new(self)?;
eprintln!(
```

```
"connect gdb by running `riscv32im-gdb -ex \"target remote {}\" {}`",
debugger.local_addr()?,
debugger.elf_path().display()
);
debugger.run()
}
}
```

10.4 테스트 및 벤치마크

10.4.1 통합 테스트

```
#[cfg(test)]
mod tests {
use risc0_zkvm::{default_prover, ExecutorEnv};

#[test]
fn  test_fibonacci() {
let  env = ExecutorEnv::builder()
.write(&10u32)
.unwrap()
.build()
.unwrap();
```

```
let prover = default_prover();
let receipt = prover.prove(FIBONACCI_ELF, &env).unwrap();

let result: u32 = receipt.journal.decode().unwrap();
assert_eq!(result, 55); // 10번째 피보나치 수
}
}
```

10.4.2 성능 벤치마크

```
// benches/fibonacci.rs
use criterion::{black_box, criterion_group, criterion_main, Criterion};

fn bench_fibonacci(c: &mut Criterion) {
c.bench_function("fibonacci_prove", |b| {
b.iter(|| {
let env = ExecutorEnv::builder()
.write(black_box(&20u32))
.unwrap()
.build()
.unwrap();

let prover = default_prover();
```

```
prover.prove(FIBONACCI_ELF, &env).unwrap()

})

});

}



criterion_group!(benches, bench_fibonacci);

criterion_main!(benches);
```

결론

RISC0는 **영지식 증명 기술을 범용 컴퓨팅에 적용한 혁신적인 플랫폼**입니다. 주요 특징은 다음과 같습니다:

핵심 혁신점

1. **RISC-V 기반**: 기존 개발 도구체인 활용 가능
2. **3계층 증명**: 확장성과 효율성의 균형
3. **모듈러 설계**: 각 컴포넌트의 독립적 최적화
4. **개발자 친화적**: 풍부한 도구와 문서

기술적 성취

- **보안**: 97비트 추정 보안 수준
- **성능**: GPU 가속으로 실용적 속도 달성
- **확장성**: 재귀 증명으로 임의 크기 계산 지원
- **검증성**: 빠르고 간단한 증명 검증

RISC0는 블록체인, 프라이버시 보호 컴퓨팅, 검증 가능한 AI 등 다양한 분야에서 **신뢰할 수 있는 컴퓨팅의 새로운 패러다임**을 제시하고 있습니다.